

Safe FSM In Xilinx

Course Workbook

Table of Contents

About this Workbook

[Slide 3](#)

Pre-Lab: Workshop Pre-requisites

[Slide 4](#)

Lab : Implementing Safe FSM In Xilinx

[Slide 7](#)

About this Workbook

This workbook is designed to be used in conjunction with the Safe FSM in Xilinx workbook.

The contents of this workbook are created by Aduvo Engineering & Training, Ltd.

If you have any questions about the contents, or need assistance, please contact Adam Taylor at adam@adiuvoengineering.com.

Pre-Lab

Workshop Pre-requisites

Required Hardware

There is no required hardware for this course.

Downloads and Installations

Step 1 – Download and install the following at least one day prior to the workshop. This may take a significant amount of time and drive space.

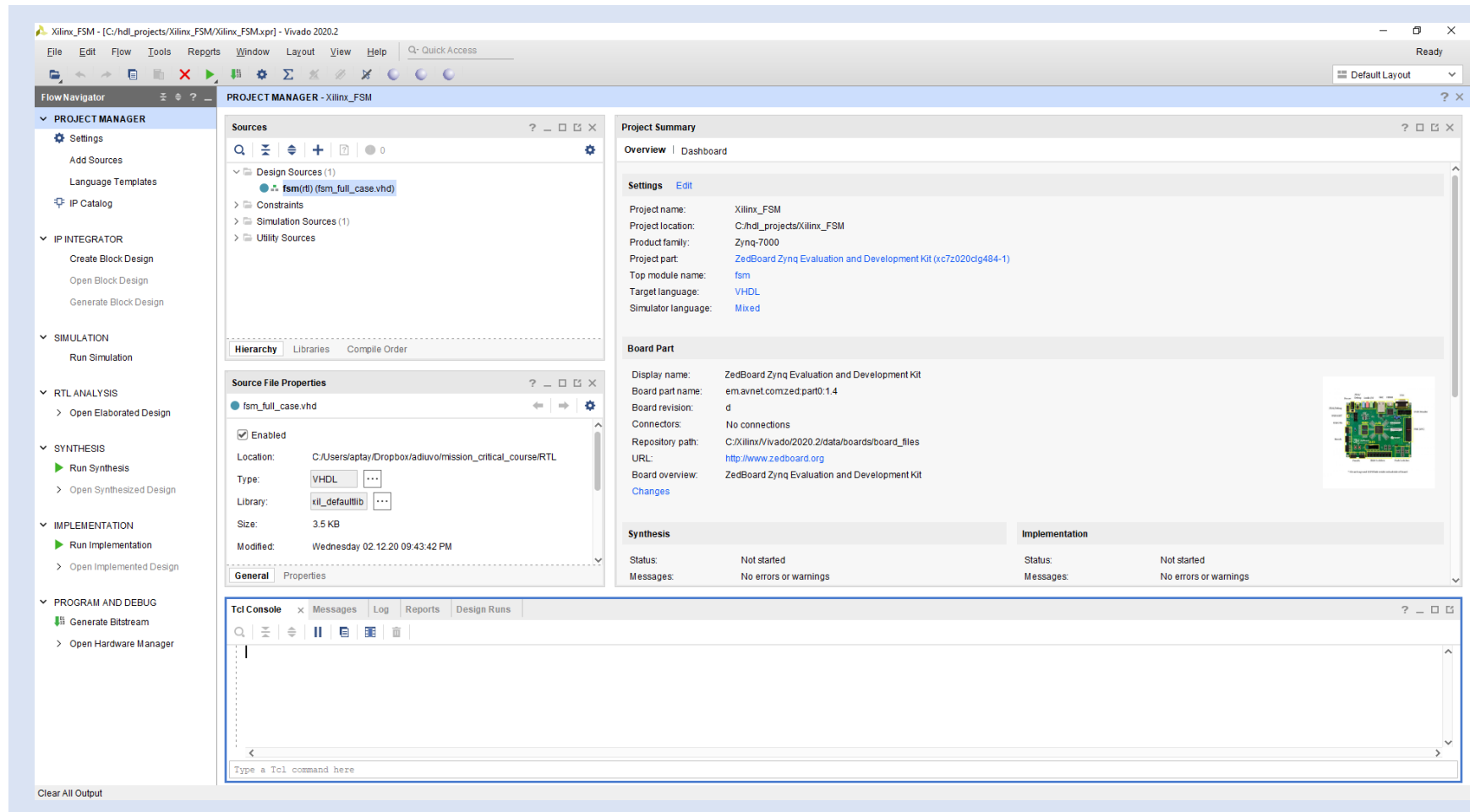
Xilinx 2020.2	Download
Source Project Files	Download

Lab

Safe FSM In Xilinx

Xilinx FSM: Safe FSM in Xilinx

Step 1 – Create a new project in Vivado 2020.2 targeting the Zed Board and import the provided files



Xilinx FSM: Overview and Introduction to Libero

Step 2 – Using Xilinx Synthesis, we can control the FSM encoding and safe FSM Implementation

FSM Encoding Options include

- One Hot
- Johnson
- Sequential
- Grey
- Auto - This is the default
- None

FSM Safe Implementation Options Include

- `auto_safe_state` – Uses Hamming 3 Encoding so a single bit error does not impact operation
- `reset_state` – Uses Hamming 2 Encoding to detect a single bit error and return to reset state
- `power_on_state` – Uses Hamming 2 Encoding to detect a single error and return to power on state
- `default_state` – Forces the state machine to the default state VHDL –When Others, Verilog Default Branch.

All controlled via attributes in either the source code or XDC

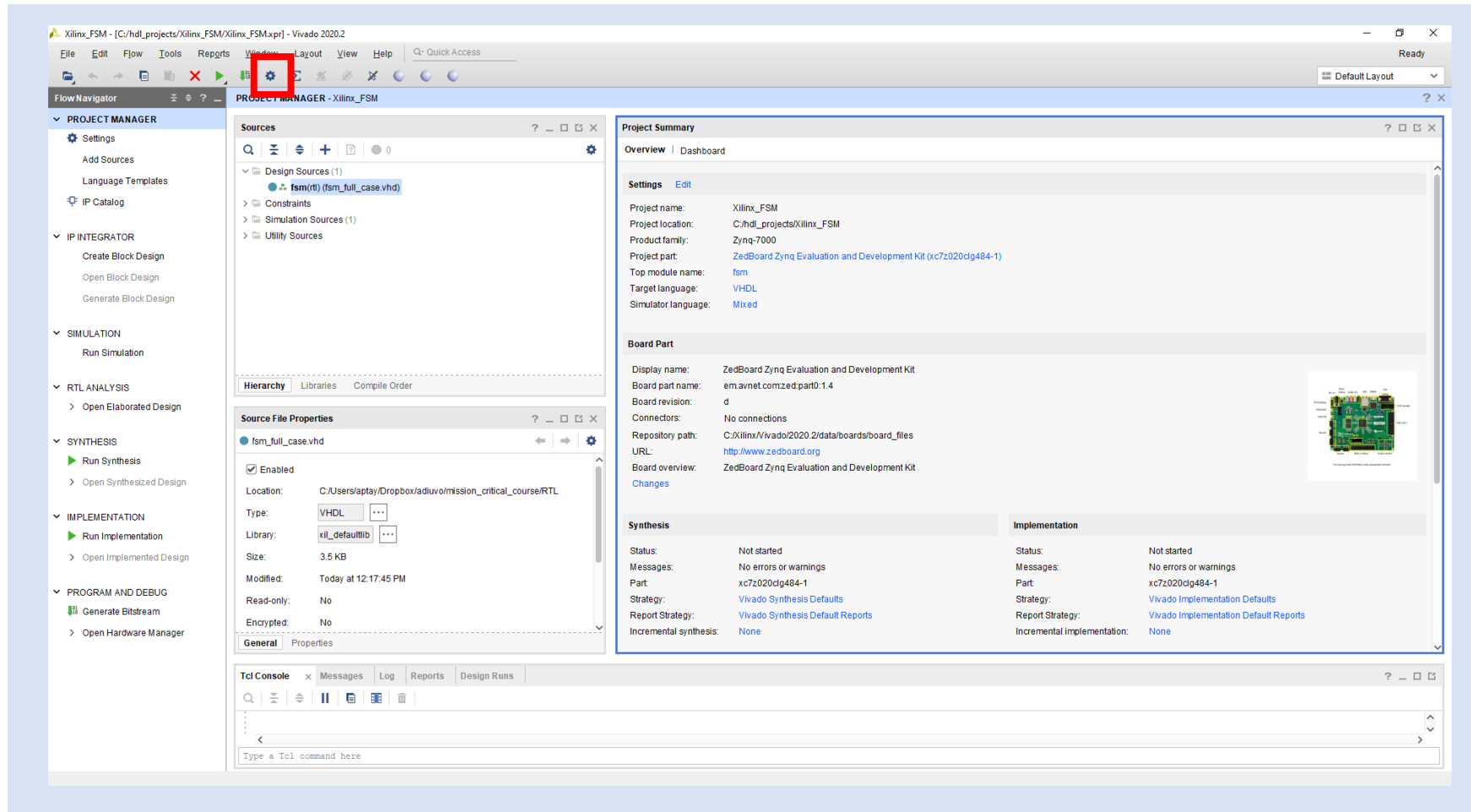
Xilinx FSM: Safe FSM in Xilinx

Step 3 – Open the FSM source file in a browser of choice and examine the RTL code note the commented-out RTL attributes on the safe state machine

```
--attribute fsm_safe_state      : string;  
--attribute fsm_safe_state of current_state : signal is "auto_safe_state";  
begin  
  
fsm : process (rst, clk)  
begin  
    if rst = '1' then  
        current_state <= idle;  
        transfer_word <= (others => '0');  
        data_out      <= (others => '0');  
    elsif rising_edge(clk) then  
        case current_state is  
            when idle =>  
                if start = '1' then  
                    if mode = '0' then  
                        current_state <= rd;  
                    else  
                        current_state <= wr;  
                    end if;  
                end if;  
            when rd =>  
                data_out      <= transfer_word;  
                current_state <= idle;  
            when wr =>  
                transfer_word <= data_in;  
                current_state <= idle;  
            when others =>  
                --current_state <= idle;  
                --null;  
            end case;  
        end if;  
    end process;
```

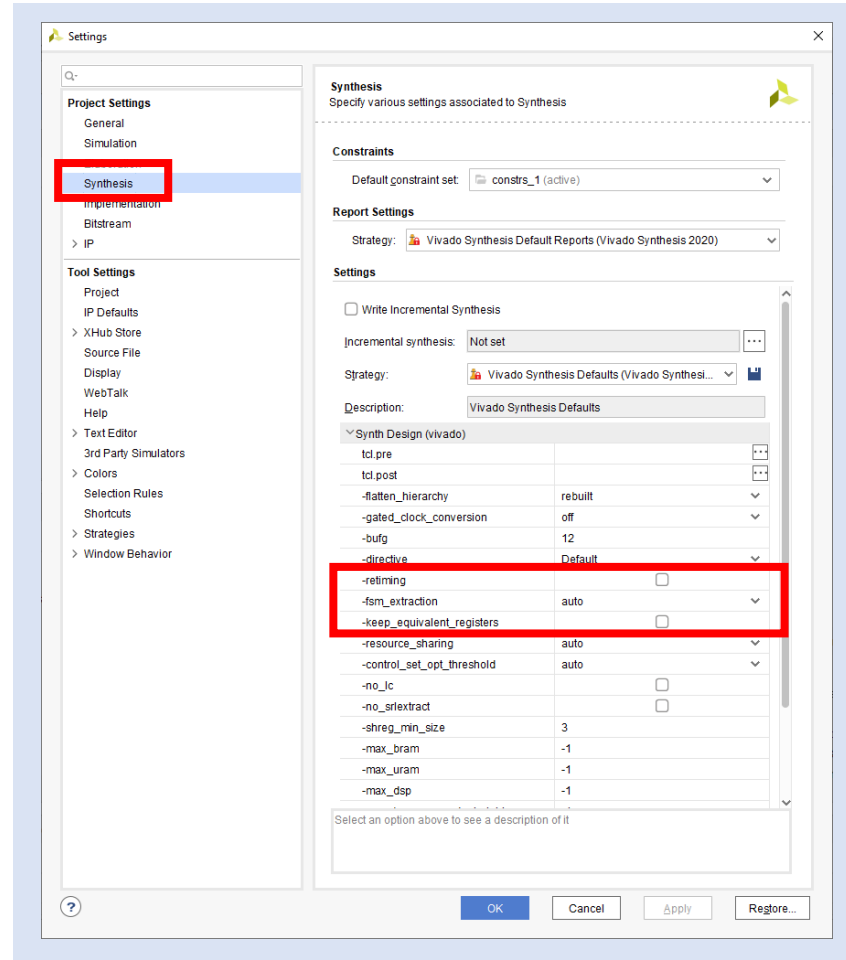
Xilinx FSM: Safe FSM in Xilinx

Step 4 – Open the XST Options and double check the encoding style is set to auto



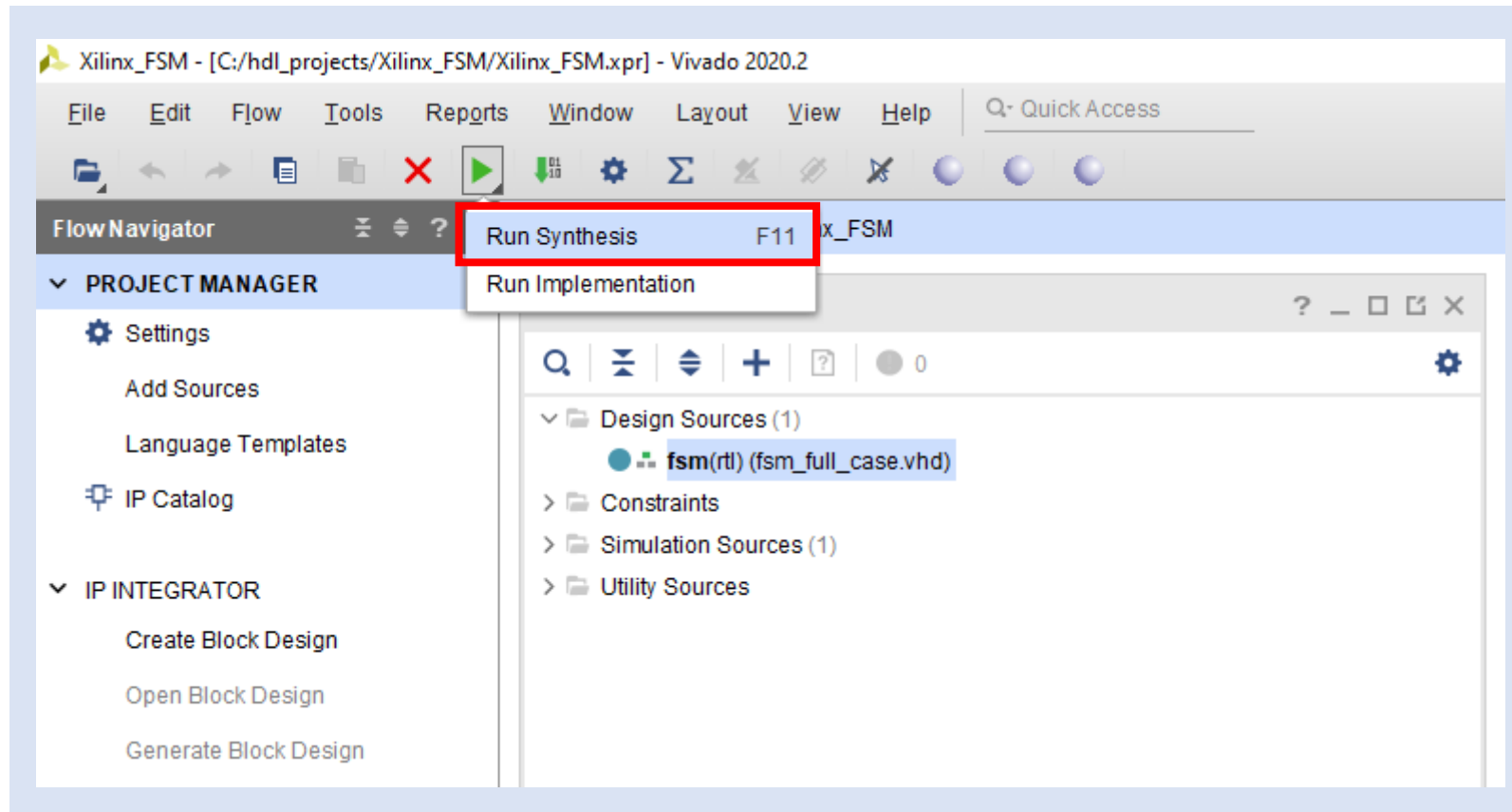
Xilinx FSM: Safe FSM in Xilinx

Step 5 – In the settings click on Synthesis settings and see the FSM extraction is set to Auto.



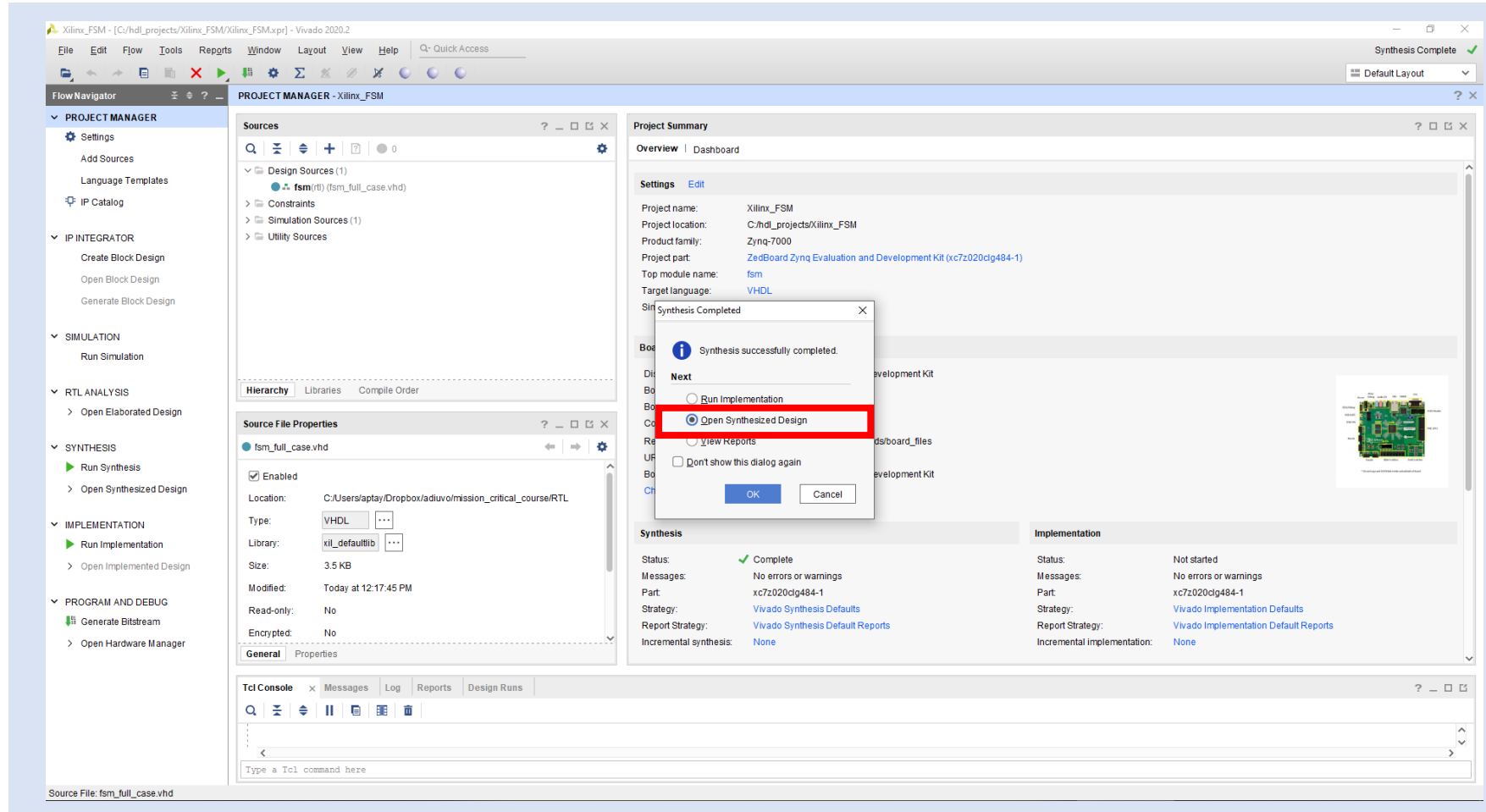
Xilinx FSM: Safe FSM in Xilinx

Step 6 – Run the Synthesis on the FSM.



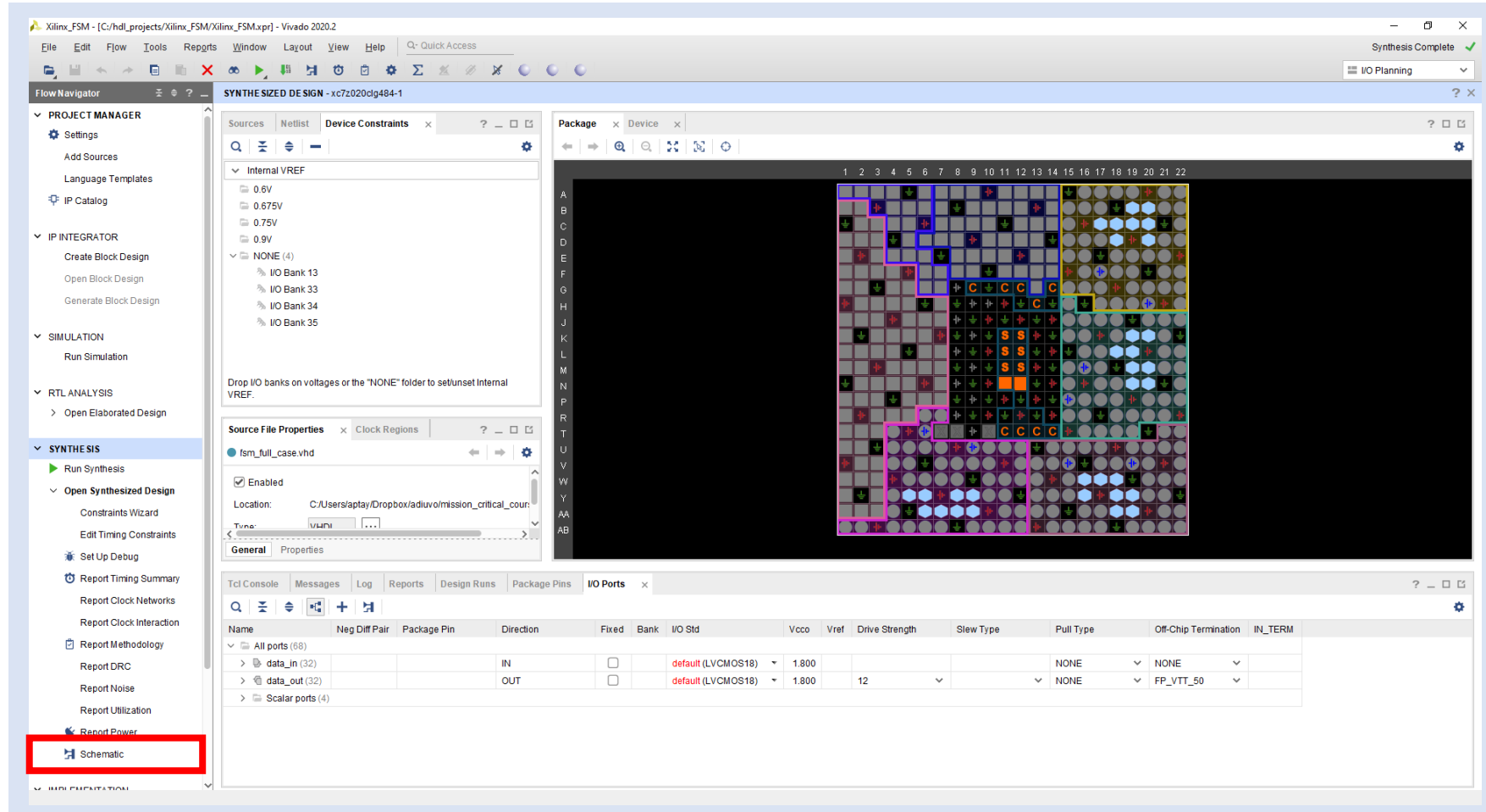
Xilinx FSM: Safe FSM in Xilinx

Step 7 – When the synthesis completed select the Open Synthesis Option.



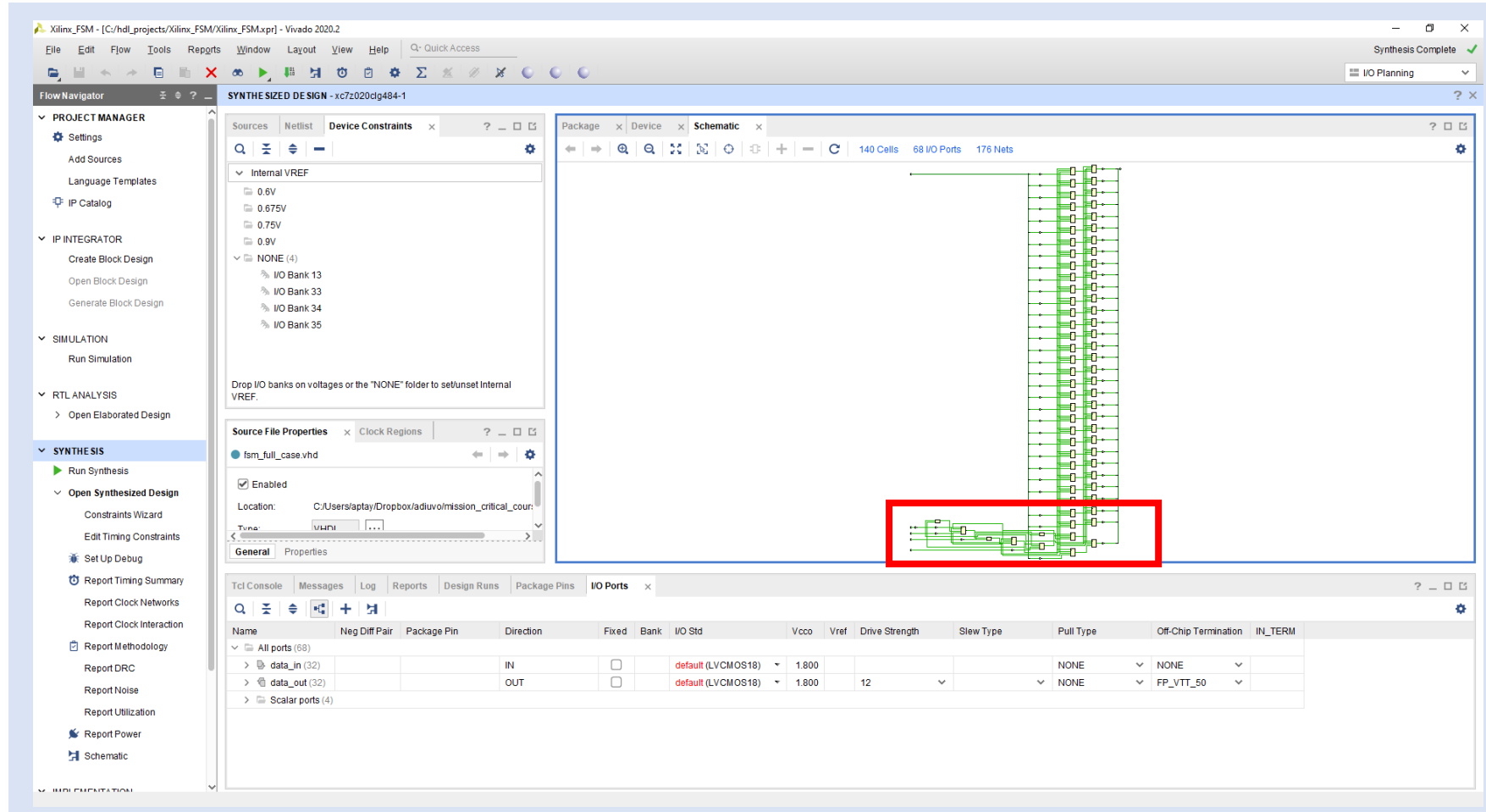
Xilinx FSM: Safe FSM in Xilinx

Step 8 – When the Synthesis view opens click on schematic to open the logic schematic.



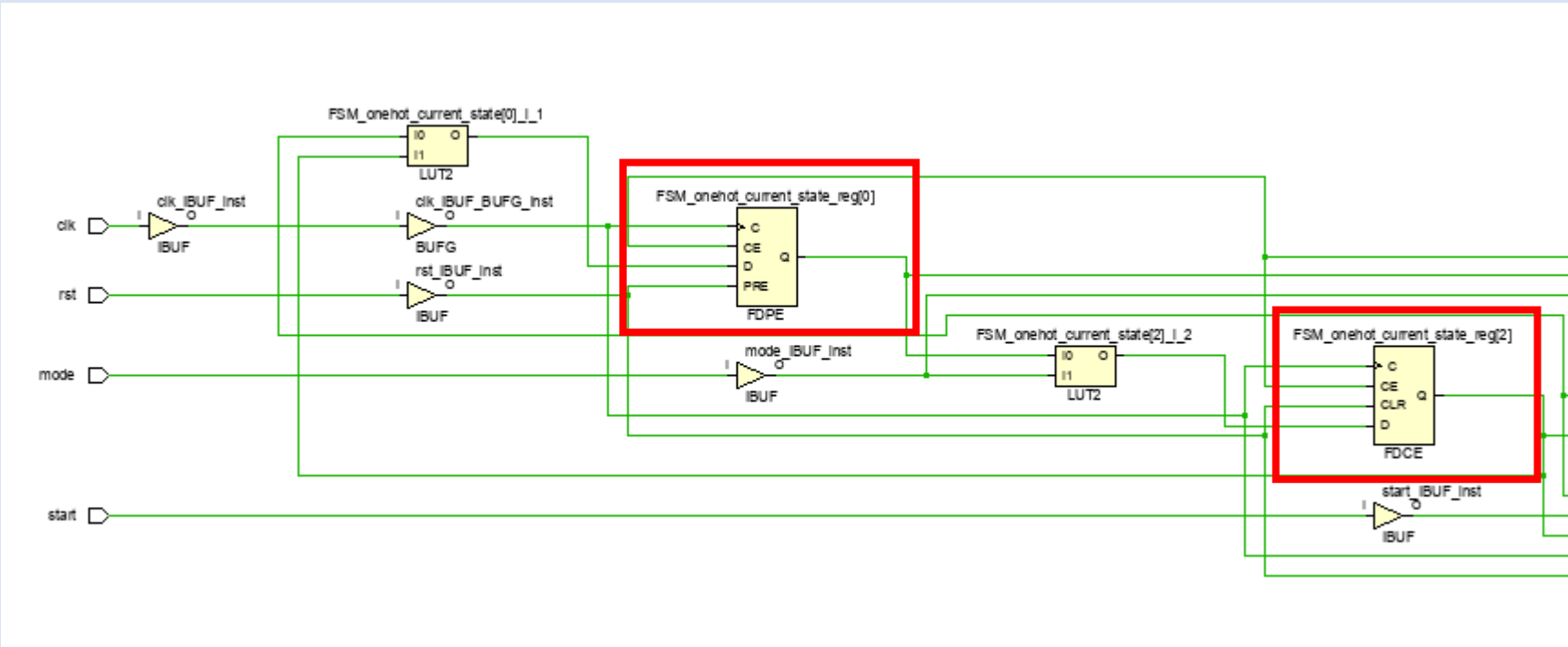
Xilinx FSM: Safe FSM in Xilinx

Step 9 – Once the schematics open, zoom in on the register area.



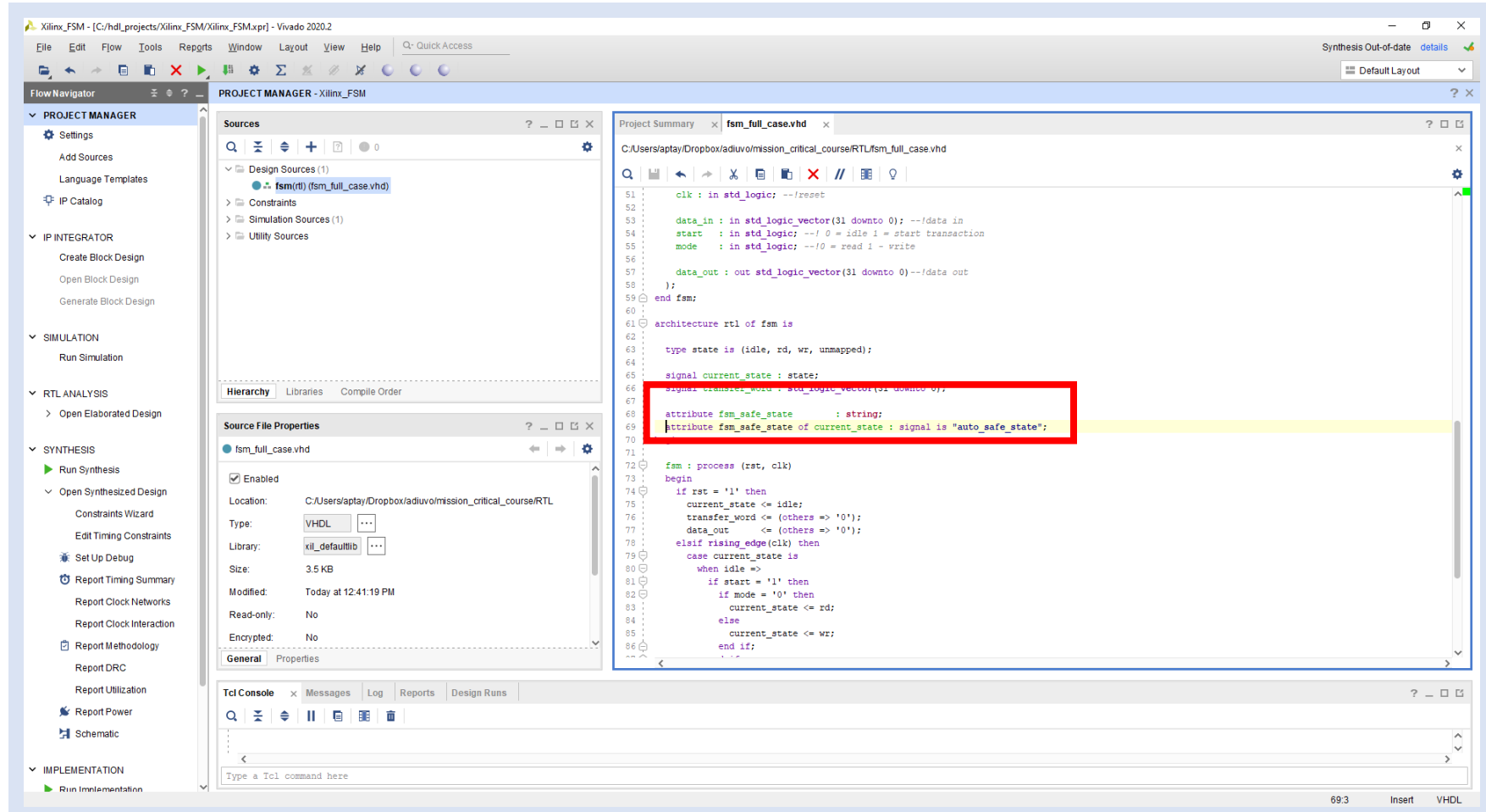
Xilinx FSM: Safe FSM in Xilinx

Step 10 – You should see the Two registers of the state machine, close the synthesis view



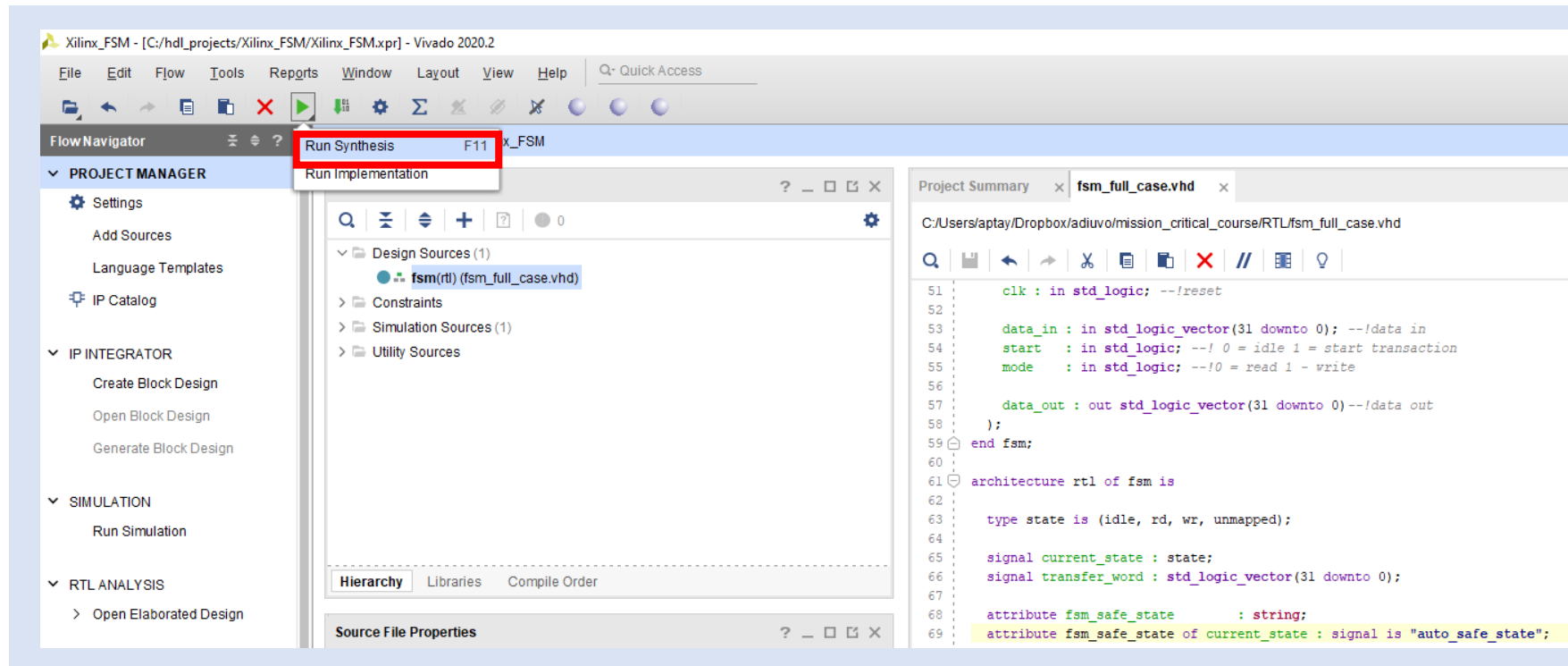
Xilinx FSM: Safe FSM in Xilinx

Step 11 – Back in the project management view, open the FSM file and uncomment the attributes. Save the file



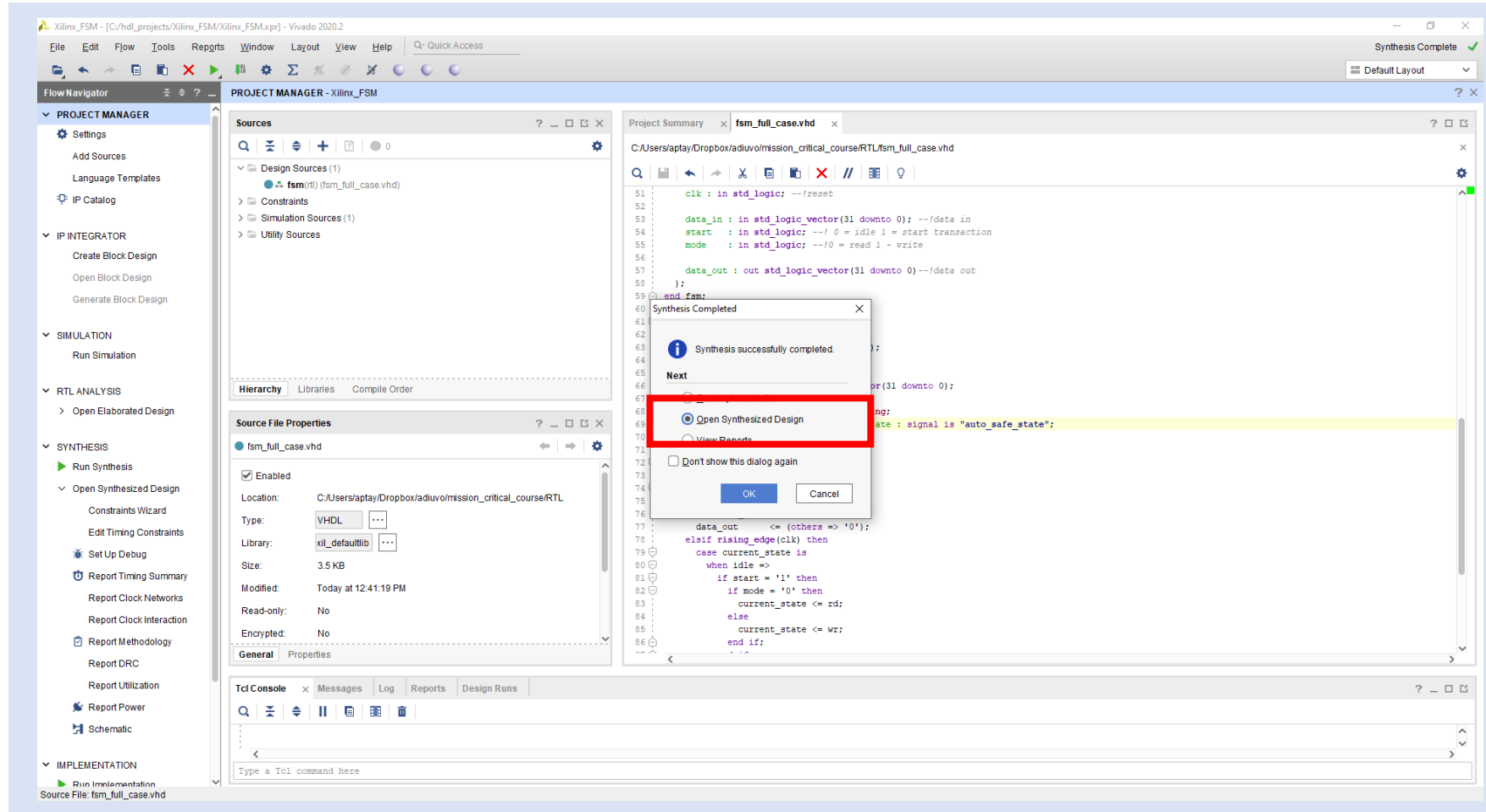
Xilinx FSM: Safe FSM in Xilinx

Step 12 – Re-Run the Synthesis.



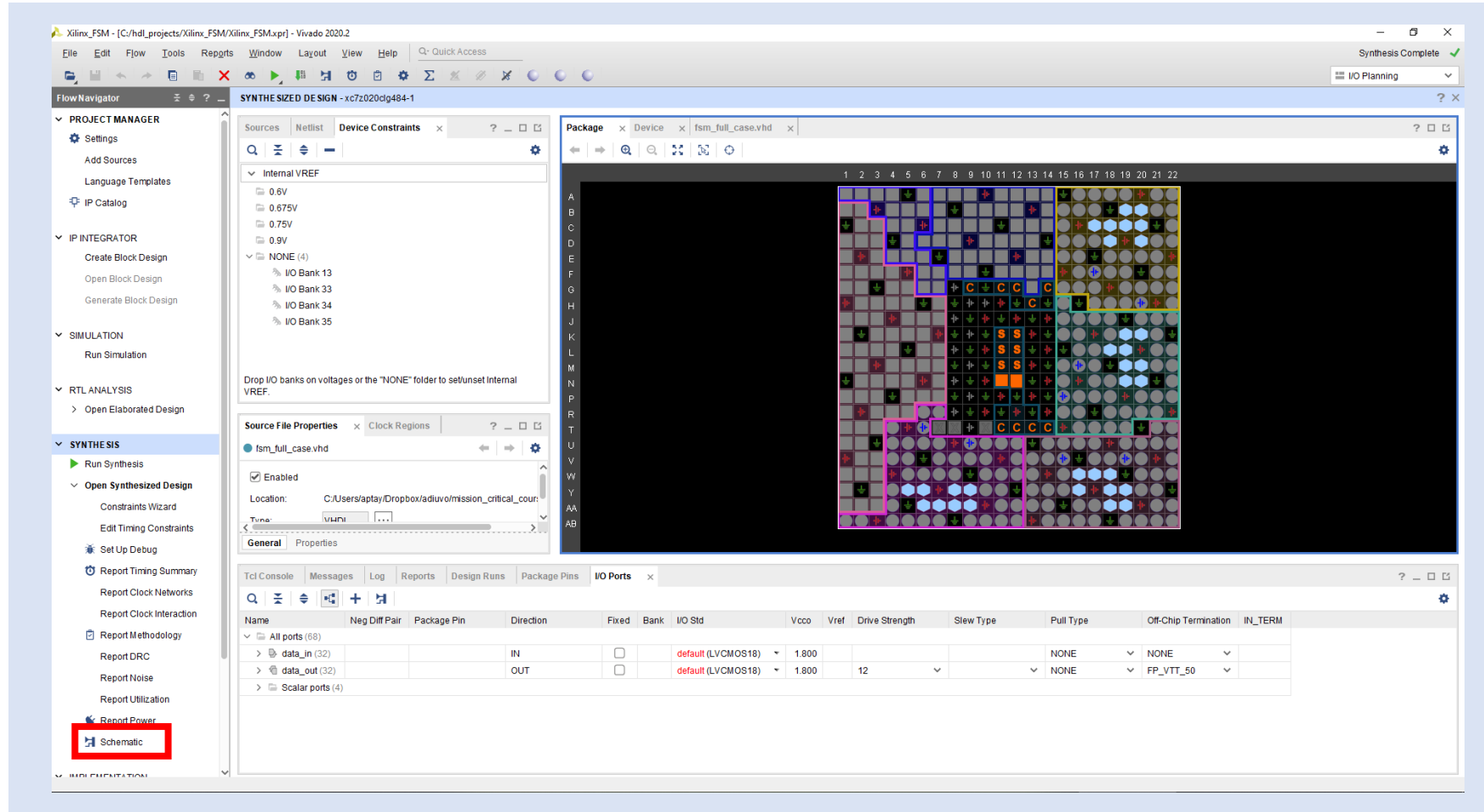
Xilinx FSM: Safe FSM in Xilinx

Step 13 – Once synthesis completes open the synthesis view.



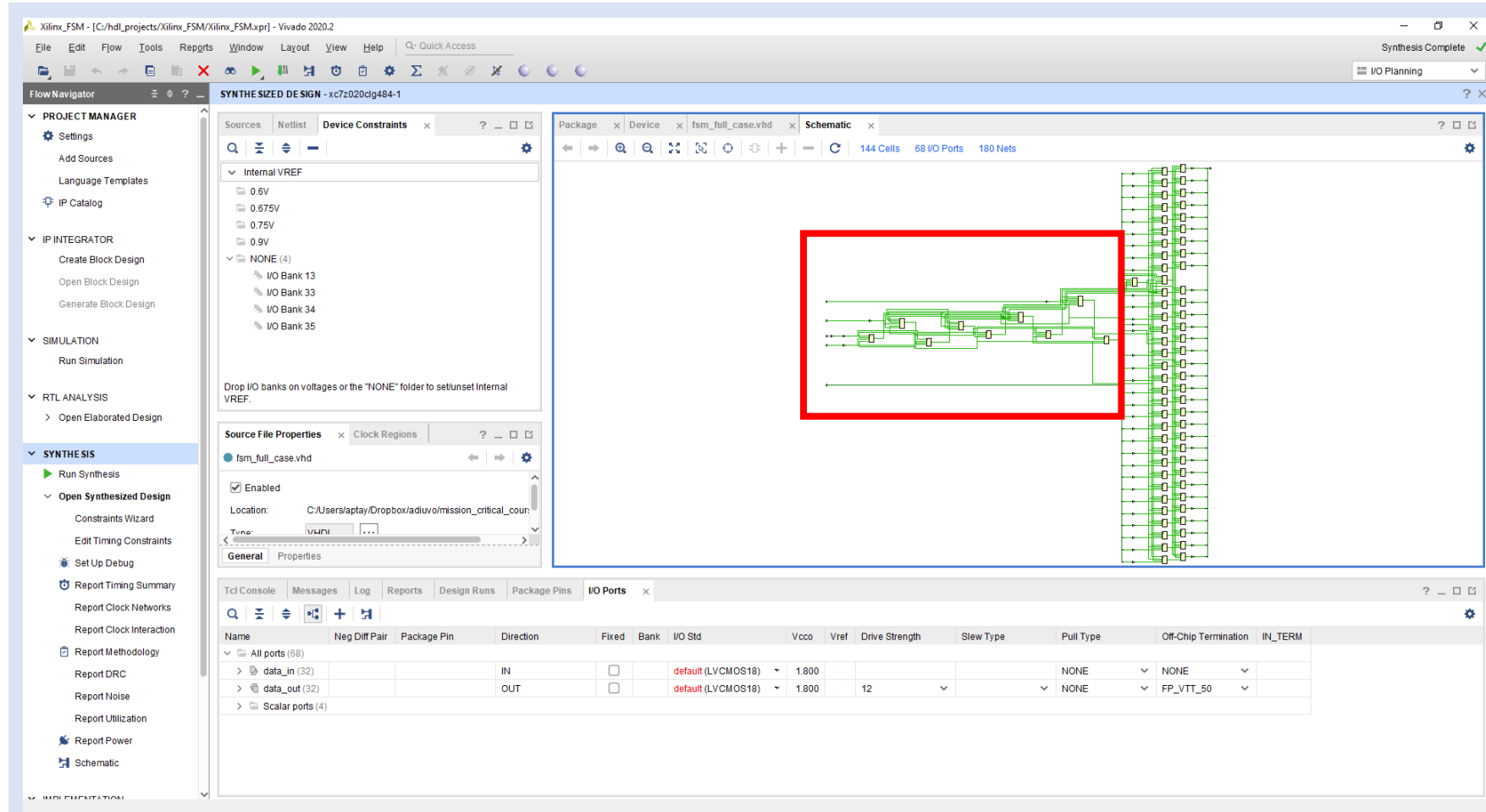
Xilinx FSM: Safe FSM in Xilinx

Step 14 – Open the schematic view.



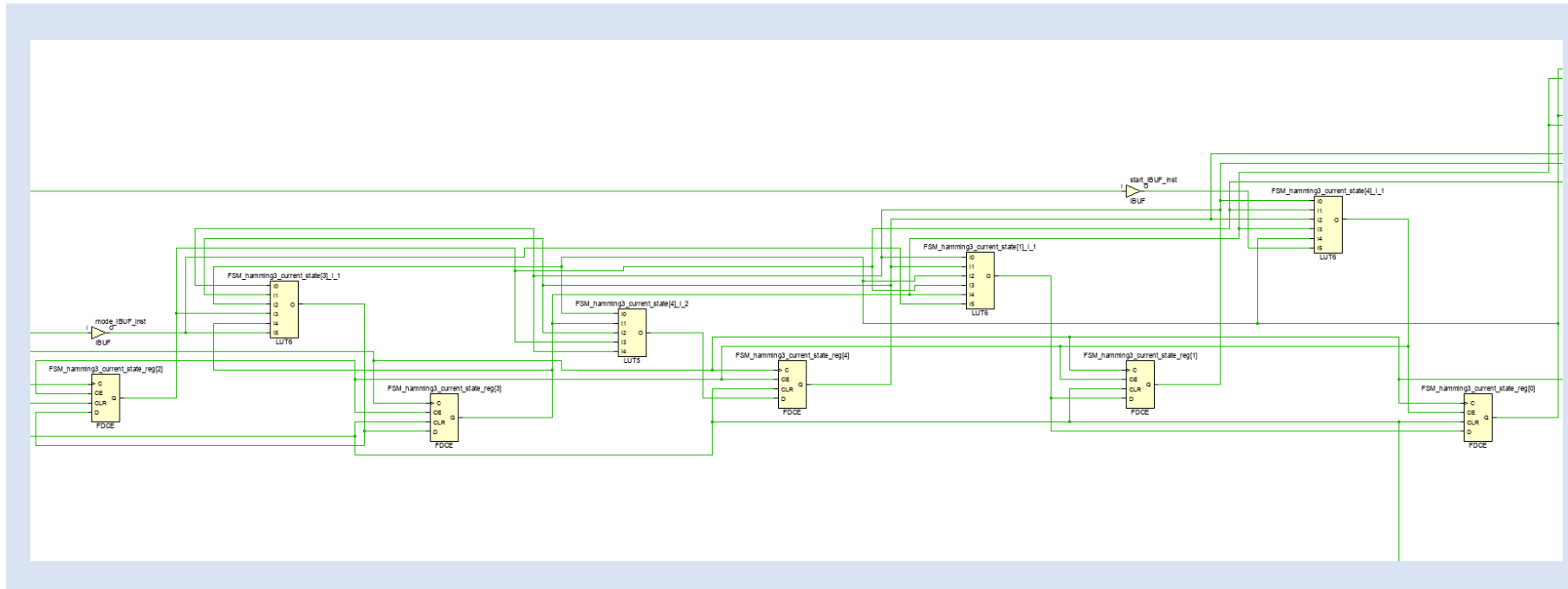
Xilinx FSM: Safe FSM in Xilinx

Step 15 – Zoom in on the FSM registers again.



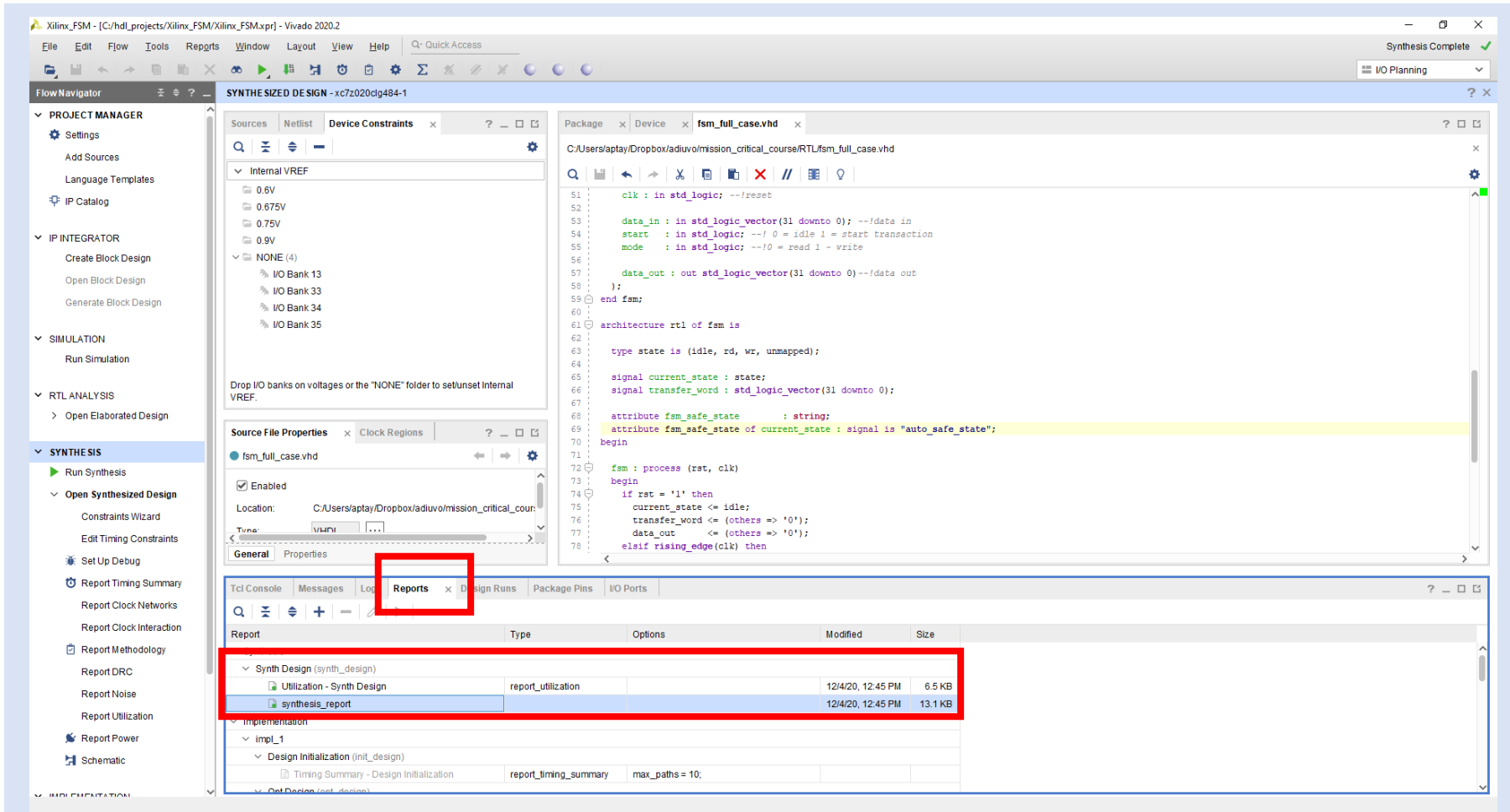
Xilinx FSM: Safe FSM in Xilinx

Step 16 – Notice the registers are marked as Hamming and there are several registers now. Close the schematic view.



Xilinx FSM: Safe FSM in Xilinx

Step 17 – Click on the reports tab and select synthesis report.



Xilinx FSM: Safe FSM in Xilinx

Step 18 – Scroll down the report until you see the FSM encoding report

```
34 -----
35 Start Loading Part and Timing Information
36 -----
37 Loading part: xc7z020clg484-1
38 -----
39 Finished Loading Part and Timing Information : Time (s): cpu = 00:00:07 ; elapsed = 00:00:08 . Memory (MB): peak = 1139.531 ; gain = 0.000
40 -----
41 INFO: [Device 21-403] Loading part xc7z020clg484-1
42 INFO: [Synth 8-802] inferred FSM for state register 'current_state_reg' in module 'fsm'
43 INFO: [Synth 8-5552] Implemented safe state 'auto_safe_state' for state register 'current_state_reg' in module 'fsm'
44 -----
45 State | New Encoding | Previous Encoding
46 -----
47 idle | 00000 | 00
48 rd | 10011 | 01
49 wr | 11100 | 10
50 -----
51 INFO: [Synth 8-3354] encoded FSM with state register 'current_state_reg' using encoding 'Hamming 3' in module 'fsm'
52 -----
53 Finished RTL Optimization Phase 2 : Time (s): cpu = 00:00:07 ; elapsed = 00:00:08 . Memory (MB): peak = 1139.531 ; gain = 0.000
```

Xilinx FSM: Safe FSM in Xilinx

Step 19 – If the Source code is suitable for attribute it can be added in the XDC file as below. Use the Elaboration view to determine the cells name if the architecture is complicated.

